NEMOX AI | RICO-7 FIELD-BASED DEVELOPER TOOL ARCHITECTURE

Engineering blueprint | 1 November 2026 | Munich, Germany | Version 1.0

One shared contextual field foundation. Three developer tools. Explicit traceability from request through decision, feedback and operational state. All architecture and code fragments in this document are independent proposals, not descriptions of nemox private internals.

PART I | EXECUTIVE OVERVIEW

1. Executive Summary

Challenge: Build Developer Tools to Better Understand Users. Logistics agents transform email and document requests into operational actions. The engineering problem is to expose interpretation, context selection, missing requirements, decision provenance, user corrections and later effects. A shared append-only event stream and versioned contextual field model supports three separate developer tools: (A) Intent & Context Inspector, (B) Recursive Feedback & Decision Trace Engine and (C) Understanding Gap Detector.

Recommended six-hour demonstration: implement the common event/field kernel and one narrow end-to-end slice of B, then show A and C as read-only projections using the same synthetic three-cycle dataset. No direct carrier booking or TMS writes are required. Success means a developer can click a decision, inspect evidence, observe a confirmed correction, and compare a later decision with and without retrieved context.

Architectural commitments: source-linked claims; immutable observations; versioned interpretations; explicit human confirmation; tenant isolation; idempotent event ingestion; replayable state transitions; deterministic evaluation fixtures; model output treated as untrusted candidate data.

2. Challenge Definition

The desired engineering output is an inspectable explanation chain: user input -> evidence -> contextual field -> requirement -> decision -> correction -> confirmed relation -> retrieval -> subsequent decision. The inspection interface must distinguish what was observed, derived, proposed and committed. Operational correctness remains with existing workflow owners.

3. Three Proposed Solutions

Tool	Primary question	Distinct output
A. Intent Inspector	What did the agent infer from this RFQ?	Evidence-linked requirement graph and unresolved references
B. Feedback Trace	Did a correction affect a later decision?	Versioned relation changes and decision comparison
C. Gap Detector	Which prerequisite blocks this action?	Dependency graph and minimal clarification set

PART II | SYSTEM FOUNDATION

4. nemox Product Context

Public product material describes Outlook/Gmail workflows, RFQ extraction, rate retrieval, margin application, quote drafting, export shipment/AWB preparation, import pre-alert processing, carrier invoice matching, TMS access, human review and activity logging. These are company claims, not inspected implementation facts [S1-S4]. Engineering hiring material emphasizes context quality and accuracy [S3]. The proposed tools integrate at observation and developer inspection

boundaries, not by presuming any existing internal service topology.

Public capability	Proposed observation boundary	Minimum correlation key
RFQ -> quote draft	email/request and quote decision adapter	request_id, quote_id
Export -> shipment/AWB	shipment state event adapter	shipment_id
Import pre-alert	document completeness adapter	shipment_id, document_id
Invoice matching	discrepancy decision adapter	invoice_id, shipment_id
Human review	approval/correction adapter	decision_id, actor_id

5. RICO-7 Reconstruction

R1 User structure

Operators, customer accounts, team roles and tenant boundaries determine which preferences are relevant.

R2 Interaction structure

Messages, attachments, agent proposals, approvals, edits and rejections are recorded as distinct events.

R3 Semantic structure

Entities and constraints are normalized with evidence spans, temporal references and explicit unknowns.

R4 Decision structure

Each candidate action stores requirements, retrieval snapshot, missing dependencies and approval state.

R5 Feedback structure

A correction is correlated to an earlier decision; a reason is stored only when explicitly supplied.

R6 Recursive calibration

Candidate relation updates are validated, reviewed, versioned and retrieved for later relevant contexts.

R7 Longitudinal understanding

Repeated corrections and superseding requirements are queryable across sessions without overwriting history.

6. Inverse Sweep

Desired developer inspection requires an immutable decision snapshot; the snapshot requires a versioned field and retrieval manifest; that field requires source-linked semantic assertions; assertions require ingestion, parsing and reference resolution. Later-effect comparison requires correction correlation, a confirmed update, retrieval logs and two decisions evaluated against equivalent inputs. Alternative path: a purely rule-based extractor can replace an LLM for the demonstration. Null path: absent source data produces an unresolved reference, never an invented shipment.

Outcome	Necessary predecessor	Failure if missing
Explain interpretation	source spans + extraction trace	unsupported inferred requirement
Explain decision	decision context snapshot	post-hoc reconstructed rationale
Show feedback effect	correction -> version -> retrieval manifest	false learning attribution
Identify blockers	dependency specification + available facts	arbitrary clarification

7. Shared Field-Based Architecture

7.1 Contextual field contract

A contextual field is a scoped, versioned collection of entities, attributes, requirements, constraints, relations, source references, temporal dependencies, workflow states and interaction/feedback events. A field is a representation for inspection and evaluation, not a claim that a human or operational system has a literal internal field.

Identity: field_id UUID; tenant_id; scope_type (user/customer/shipment/agent/workflow/external/developer); scope_ref; version integer; state enum; valid_from; recorded_at; provenance references. Relations have subject, predicate, object, source event, confidence status (explicit/derived/unresolved), and temporal validity. Every projection is rebuildable from events.

7.2 Boundaries and permitted crossings

Boundary	Crossing contract	Restriction
Email -> request	message_id, attachment refs, sender scope	redact secrets; scoped access
Customer -> shipment	customer_id + shipment_id relation	no cross-tenant join
Agent -> decision	model run, prompt template version, inputs, output	candidate only
Workflow -> field	status event and source version	do not mutate external TMS
Feedback -> relation	decision_id, actor, correction, review state	confirmation required
Field -> developer UI	authorized projection with evidence refs	read-only inspection by default

7.3 Coupling and state

Coupling is an explicit relation established by a source-backed event: REQUEST_REFERENCES_SHIPMENT, SHIPMENT_SUPPORTS_CARRIER, DECISION_CORRECTED_BY, FEEDBACK_RETRIEVED_FOR. Enforce referential integrity, scope compatibility and temporal validity before activation. Field lifecycle: INITIAL -> ENRICHED -> UNRESOLVED -> CORRECTED -> CONFIRMED -> SUPERSEDED. ENRICHED can transition to CONFIRMED; UNRESOLVED can transition to ENRICHED after clarification; CORRECTED must be reviewed before CONFIRMED; SUPERSEDED is terminal for that version, not for the field identity.

7.4 Event and consistency protocol

Ingestion transaction: verify tenant and idempotency key; insert immutable event; persist outbox job; return event_id. Projection worker: lock stream, validate expected_version, apply transition, persist new field version and processed offset atomically. Duplicate key returns original response. Out-of-order event enters pending reconciliation queue. Concurrent updates use compare-and-swap on field_version. A failed model call records extraction_failed and leaves operational state unchanged.

PART III | SOLUTION ARCHITECTURES

8. Solution A: User Intent & Context Inspector

Target workflow: RFQ and quotation. The tool operates as an independent developer-facing inspection projection over the shared field kernel. It does not issue real logistics actions in the prototype.

Field construction and processing

Fields: CustomerContext, ShipmentRequirements, HistoricalReference, OperationalConstraints. Processing pipeline: ingest -> parse -> extract -> resolve references -> couple fields -> flag ambiguity -> inspect.

Component contract matrix

Component	Input -> output	Failure behavior
Input Adapter	email/document refs -> UserRequest	source unavailable -> queued
Document Parser	bytes/ref -> text segments with offsets	parse_failed, retain original
Context Resolver	reference query -> candidate context refs	ambiguous -> unresolved
Requirement Extractor	segments + context -> typed requirements	schema invalid -> retry then failure
Field Relation Builder	validated facts -> semantic relations	reject cross-tenant edges
Interpretation Trace Service	evidence + requirements -> trace	mark missing evidence
Developer Dashboard	trace projection -> relation UI	show partial result

Data models and endpoint contracts

Endpoint	Request -> response
POST /v1/requests	UserRequest -> RequestReceipt
POST /v1/interpretations	request_id -> InterpretationTrace
GET /v1/fields/{id}	field_id -> ContextField
GET /v1/relations	field_id -> SemanticRelation[]
GET /v1/unresolved	request_id -> UnresolvedReference[]

All endpoints require tenant authorization; POST requires Idempotency-Key and correlation_id. Validation errors 422, unknown resource 404, scope mismatch 403, version conflict 409, unavailable upstream 503 with retryable flag. Responses contain schema_version, request_id, recorded_at and source_refs; list endpoints use cursor pagination.

Demonstration and acceptance

Synthetic demonstration: Same shipment as last week, but deliver two days earlier.

Expected: shipment reference unresolved until a synthetic historical shipment is selected; delivery change stored as relative temporal constraint with source span; no absolute delivery date inferred without baseline.

Acceptance: every displayed assertion has a source_ref or unresolved marker; API schema validates; duplicate ingestion is idempotent; tenant-crossing references are rejected; deterministic fixture reproduces expected output.

Implementation work packages

A1: Input Adapter. Owner: Developer A. Input/output: email/document refs -> UserRequest. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: source unavailable -> queued. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

A2: Document Parser. Owner: Developer A. Input/output: bytes/ref -> text segments with offsets. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: parse_failed, retain original. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

A3: Context Resolver. Owner: Developer A. Input/output: reference query -> candidate context refs. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: ambiguous -> unresolved. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

A4: Requirement Extractor. Owner: Developer A. Input/output: segments + context -> typed requirements. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: schema invalid -> retry then failure. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

A5: Field Relation Builder. Owner: Developer A. Input/output: validated facts -> semantic relations. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: reject cross-tenant edges. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

A6: Interpretation Trace Service. Owner: Developer B. Input/output: evidence + requirements -> trace. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: mark missing evidence. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

A7: Developer Dashboard. Owner: Developer B. Input/output: trace projection -> relation UI. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: show partial result. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

9. Solution B: Recursive Feedback & Decision Trace Engine

Target workflow: carrier selection, quote correction and approval. The tool operates as an independent developer-facing inspection projection over the shared field kernel. It does not issue real logistics actions in the prototype.

Field construction and processing

Fields: DecisionContext, Feedback, CorrectionRelation, HistoricalContext, SubsequentDecision. Processing pipeline: capture decision -> snapshot -> correlate feedback -> propose relation -> confirm -> retrieve -> compare.

Component contract matrix

Component	Input -> output	Failure behavior
Decision Trace Collector	agent output + context -> immutable snapshot	reject missing correlation
Feedback Event Processor	user edit -> typed correction	unmatched -> pending
Relation Update Service	correction + scope -> candidate relation	requires explicit confirmation
Field State Store	event + version -> transition	optimistic concurrency conflict
Context Retrieval Service	scope query -> ranked confirmed relations	return retrieval manifest
Decision Comparison Engine	two decision traces -> structured diff	different inputs flagged
Developer Timeline	events + states -> inspectable chronology	partial history indicator

Data models and endpoint contracts

Endpoint	Request -> response
POST /v1/decisions	AgentDecision -> DecisionReceipt
POST /v1/feedback	FeedbackEvent -> CandidateCorrection
GET /v1/corrections	decision_id -> CorrectionRelation[]
POST /v1/context/retrieve	ContextQuery -> RetrievedContext
POST /v1/decisions/compare	DecisionPair -> DecisionComparison

Endpoint	Request -> response
GET /v1/transitions	field_id -> FieldStateTransition[]

All endpoints require tenant authorization; POST requires Idempotency-Key and correlation_id. Validation errors 422, unknown resource 404, scope mismatch 403, version conflict 409, unavailable upstream 503 with retryable flag. Responses contain schema_version, request_id, recorded_at and source_refs; list endpoints use cursor pagination.

Demonstration and acceptance

Synthetic demonstration: Cycle 1: carrier proposal; cycle 2: explicit carrier correction; cycle 3: similar RFQ reuses confirmed preference.

Expected: initial decision remains immutable; correction creates candidate relation; explicit confirmation increments field version; third decision retrieval manifest points to confirmed relation; compare view shows both decisions and their differing context.

Acceptance: every displayed assertion has a source_ref or unresolved marker; API schema validates; duplicate ingestion is idempotent; tenant-crossing references are rejected; deterministic fixture reproduces expected output.

Implementation work packages

B1: Decision Trace Collector. Owner: Developer A. Input/output: agent output + context -> immutable snapshot. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: reject missing correlation. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

B2: Feedback Event Processor. Owner: Developer A. Input/output: user edit -> typed correction. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: unmatched -> pending. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

B3: Relation Update Service. Owner: Developer A. Input/output: correction + scope -> candidate relation. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: requires explicit confirmation. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

B4: Field State Store. Owner: Developer A. Input/output: event + version -> transition. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: optimistic concurrency conflict. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

B5: Context Retrieval Service. Owner: Developer A. Input/output: scope query -> ranked confirmed relations. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: return retrieval manifest. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

B6: Decision Comparison Engine. Owner: Developer B. Input/output: two decision traces -> structured diff. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: different inputs flagged. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

B7: Developer Timeline. Owner: Developer B. Input/output: events + states -> inspectable chronology. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: partial history indicator. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

10. Solution C: User Understanding Gap Detector

Target workflow: booking, export and import. The tool operates as an independent developer-facing inspection projection over the shared field kernel. It does not issue real logistics actions in the prototype.

Field construction and processing

Fields: Request, RequiredInformation, AvailableContext, MissingDependency, Clarification. Processing pipeline: ingest -> extract -> load workflow dependency DAG -> check availability -> compute blockers -> clarify -> re-evaluate.

Component contract matrix

Component	Input -> output	Failure behavior
Requirement Dependency Resolver	workflow template -> required attributes DAG	cycle -> invalid template
Context Availability Checker	required attribute + context -> available/unknown	stale -> unavailable
Gap Detection Engine	DAG + availability -> blocking gaps	unknown -> unresolved
Clarification Generator	minimal gaps -> source-specific question	no guessing values
Context Completion Service	answer -> sourced attribute + re-evaluation	invalid answer remains open
Developer Gap Inspector	dependency projection -> blocking-path view	display unresolved edges

Data models and endpoint contracts

Endpoint	Request -> response
POST /v1/dependencies/analyze	DependencyAnalysisRequest -> GapReport
GET /v1/gaps	request_id -> UnderstandingGap[]
POST /v1/clarifications	gap_ids -> ClarificationRequest
POST /v1/gaps/{id}/resolve	GapResolution -> ResolutionReceipt
GET /v1/fields/{id}/completeness	field_id -> CompletionReport

All endpoints require tenant authorization; POST requires Idempotency-Key and correlation_id. Validation errors 422, unknown resource 404, scope mismatch 403, version conflict 409, unavailable upstream 503 with retryable flag. Responses contain schema_version, request_id, recorded_at and source_refs; list endpoints use cursor pagination.

Demonstration and acceptance

Synthetic demonstration: Ship this as usual, but use the cheaper option.

Expected: cheaper option requires baseline shipment, rate options and delivery limit; generator asks the smallest blocking question; a valid answer updates availability and re-runs the graph.

Acceptance: every displayed assertion has a source_ref or unresolved marker; API schema validates; duplicate ingestion is idempotent; tenant-crossing references are rejected; deterministic fixture reproduces expected output.

Implementation work packages

C1: Requirement Dependency Resolver. Owner: Developer A. Input/output: workflow template -> required attributes DAG. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: cycle -> invalid template. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

C2: Context Availability Checker. Owner: Developer A. Input/output: required attribute + context -> available/unknown. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: stale -> unavailable. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

C3: Gap Detection Engine. Owner: Developer A. Input/output: DAG + availability -> blocking gaps. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field

schema; for later stages, preceding component output. Failure: unknown -> unresolved. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

C4: Clarification Generator. Owner: Developer A. Input/output: minimal gaps -> source-specific question. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: no guessing values. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

C5: Context Completion Service. Owner: Developer B. Input/output: answer -> sourced attribute + re-evaluation. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: invalid answer remains open. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

C6: Developer Gap Inspector. Owner: Developer B. Input/output: dependency projection -> blocking-path view. Implement a typed service interface, persist event references and expose a fixture-driven unit test. Dependency: shared event envelope and field schema; for later stages, preceding component output. Failure: display unresolved edges. Acceptance: valid fixture yields typed output; invalid fixture yields typed error; no external write.

Representative mechanism

```
def blockers(required, available, edges):
    missing = set(required) - set(available)
    return {x for x in missing if all(y in available for y in edges.get(x, []))}
```

PART IV | IMPLEMENTATION

11. Shared Technical Platform

Backend: FastAPI routing; Pydantic DTO validation; SQLAlchemy repositories; SQLite in local WAL mode and PostgreSQL in extended deployment; transactional outbox; background projector; provider-independent extraction adapter. Frontend: React/TypeScript/Vite, query cache, timeline, decision diff, React Flow field graph. Split modules: api, domain, services, adapters, persistence, evaluation. Dependency rule: adapters depend on domain ports; domain never imports infrastructure.

Data ownership: events are append-only; projection tables are derived; model extraction records are immutable candidate artifacts. Production adapters use scoped credentials and do not pass raw customer data to an unapproved model endpoint. The demo uses synthetic data and mock TMS/email connectors.

12. Data Models and API Contracts

Entity	Required identifiers / critical fields	Invariant
EventEnvelope	event_id, tenant_id, type, aggregate_id, occurred_at, recorded_at, correlation_id, idempotency_key, source_refs	immutable
ContextField	field_id, scope, scope_ref, version, state, relation_ids	version monotonic
Requirement	requirement_id, kind, value, unit, evidence_ref, status	typed value
SemanticRelation	relation_id, subject_id, predicate, object_id, provenance	scope compatible
AgentDecision	decision_id, request_id, context_version, action, snapshot	snapshot immutable
FeedbackEvent	feedback_id, decision_id, actor_id, correction, explicit_reason	correlation mandatory
StateTransition	transition_id, from_state, to_state, from_version, to_version, cause_event_id	valid edge
UnderstandingGap	gap_id, required_attribute, blocking_dependencies, status	resolved only with evidence

Canonical identifiers are UUID strings. All timestamps use RFC 3339 UTC. Source references contain source_type, source_id, location (e.g. document segment offsets), hash and tenant_id. The server derives tenant_id from authentication,

never trusts it from client input. Unknown and not-applicable are distinct values.

13. Implementation Techniques

Extraction: segment document; pass evidence-tagged text to a provider adapter; demand strict structured JSON; validate Pydantic; reject unsupported field names; link each assertion to evidence; retry only transient transport errors and schema failures within a bounded budget. Do not convert model confidence to calibrated probability.

Retrieval: filter confirmed relations by tenant, customer, workflow, validity window and relevance; log candidate IDs, selected IDs and exclusion reasons; pass a bounded manifest to the agent. Feedback: correlate by decision_id, compare explicit old/new values, create a proposed relation update, require confirmation, emit FieldTransition, then make the confirmed relation eligible for retrieval.

Dependency detection: maintain a directed acyclic graph of prerequisites per workflow action; evaluate required attributes against fresh context; compute missing leaf prerequisites and minimal clarification candidates. Cycle detection rejects invalid templates.

14. Representative Code

```
from pydantic import BaseModel, Field
from typing import Literal
from datetime import datetime
from uuid import UUID

class SourceRef(BaseModel):
    source_id: str
    location: str | None = None

class FieldEvent(BaseModel):
    event_id: UUID
    correlation_id: UUID
    occurred_at: datetime
    kind: Literal["request", "decision", "feedback", "confirmation"]
    source_refs: list[SourceRef] = Field(default_factory=list)
```

```
export type RelationEdge = {
  id: string; source: string; target: string;
  predicate: string; status: 'explicit'|'derived'|'unresolved';
  sourceRefs: string[]; fieldVersion: number;
};
```

```
async def extract_requirements(adapter, segments):
    response = await adapter.generate_json(
        template_id="logistics_requirements_v1",
        payload={"segments": segments})
    result = RequirementBatch.model_validate(response)
    ensure_evidence_spans_exist(result, segments)
    return result
```

Nonproprietary extraction prompt examples

Requirements template: "From the provided numbered text segments, extract shipment entities, requested changes, constraints and references. Return JSON matching the supplied schema. Every extracted value must cite a segment ID. Mark absent information unresolved. Do not infer historical shipment values."

Correction template: "Given an immutable decision record and a user response, identify explicitly changed fields. Return old_value, new_value, evidence segment, decision_id and optional explicitly stated reason. Do not create a persistent preference unless a separate confirmation event exists."

15. Two-Developer Task Allocation

Developer A	Developer B	Handoff
Event DTOs, field model, migrations	Vite shell, API client, mock views	OpenAPI schema and fixture JSON
Ingestion, extraction, relation updates	Interaction explorer, graph, timeline	GET projection contracts
State machine, retrieval, diff	Decision comparison, gap inspector	comparison JSON fixtures
API/integration tests	UI tests and demo narration	joint smoke test

16. Six-Hour Development Roadmap

Time	Developer A	Developer B	Gate
0:00-1:00	agree schemas, event model	wireframes, fixture UX	contract frozen
1:00-2:00	FastAPI, SQLite, migrations	Vite shell, typed API client	health + seed
2:00-3:00	one extraction path + relation store	request/decision panels	trace visible
3:00-4:00	feedback correlation + confirmation	timeline + relation graph	three-cycle fixture
4:00-5:00	retrieval + comparison + tests	integration and UI smoke	end-to-end path
5:00-6:00	fix defects, demo API	demo narrative, fallback screenshots	repeatable demo

MVP A: one synthetic RFQ, typed requirements, one unresolved historical reference. MVP B: three cycles, one confirmed correction, retrieval manifest and before/after diff. MVP C: one dependency DAG, two missing attributes and one resolved clarification. If integration slips, demonstrate B as the full vertical slice and show A/C using precomputed, schema-valid projections. Blocking dependencies: agreed IDs, event envelope, source refs, fixture schema and API error contract.

17. Testing and Acceptance Criteria

Test	Fixture	Expected result
Extraction	historical shipment + relative delivery date	historical ref unresolved; delta = -2 days
Context resolution	two candidate shipments	ambiguous, no auto-selection
Relation integrity	cross-tenant edge	rejected
Feedback correlation	unknown decision_id	pending/unmatched, no update
State transition	UNRESOLVED -> CONFIRMED without evidence	rejected
Gap detection	missing shipment and delivery limit	both blockers reported
Idempotency	same event key twice	one stored event
Retrieval	confirmed vs proposed carrier preference	confirmed only
UI	click decision in three-cycle timeline	evidence and state version visible

These are architectural test specifications; this document does not assert that the proposed full platform or its UI tests were executed. Run unit tests with pytest, API contract tests against generated OpenAPI, and Playwright UI tests against seeded synthetic data. Gate on stable fixtures, explicit failures and replay consistency.

PART V | NEMOX PRODUCT IMPROVEMENT

18. Quotation Improvements

Publicly described flow: RFQ -> rate lookup -> margin -> draft -> review. Potential architectural gap to investigate: historical RFQ references and repeated edits may be disconnected. Proposed inspection components: A + B. Required data: request, quote, customer, decision, edit. Adapter emits source-linked observations; field projection joins only authorized references; developer UI shows missing or corrected relations. Evaluation: unresolved references per RFQ; correction recurrence. No measured improvement is asserted.

19. Export Improvements

Publicly described flow: booking -> TMS shipment -> AWB/documents -> human action. Potential architectural gap to investigate: document prerequisites may cross workflow boundaries. Proposed inspection components: C + A. Required data: booking, shipment, AWB fields, approval. Adapter emits source-linked observations; field projection joins only authorized references; developer UI shows missing or corrected relations. Evaluation: missing prerequisites before draft; clarification turns. No measured improvement is asserted.

20. Import Improvements

Publicly described flow: pre-alert -> extraction -> document preparation. Potential architectural gap to investigate: incomplete shipment references may delay resolution. Proposed inspection components: C. Required data: pre-alert segments, shipment lookup, document requirements. Adapter emits source-linked observations; field projection joins only authorized references; developer UI shows missing or corrected relations. Evaluation: gap resolution time; missing-reference rate. No measured improvement is asserted.

21. Accounting Improvements

Publicly described flow: invoice extraction -> shipment match -> discrepancy flag. Potential architectural gap to investigate: repeated mismatch handling may lose prior correction context. Proposed inspection components: B + C. Required data: invoice line, shipment line, discrepancy, reviewer edit. Adapter emits source-linked observations; field projection joins only authorized references; developer UI shows missing or corrected relations. Evaluation: repeat discrepancy rate; decision trace coverage. No measured improvement is asserted.

22. Cross-Workflow Architecture

Proposed chain: Customer -> Quote -> Booking -> Shipment -> Invoice -> Feedback. Each transition emits an entity-link event with source system, external ID, tenant, event time and field version. A link registry resolves references without copying entire external records. A workflow transfer manifest lists which constraints remain active, which have expired, and which require revalidation. Missing links become inspectable gaps; conflicting identifiers remain unresolved until a source-backed correction.

PART VI | FINAL ARCHITECTURE

23. Architecture Comparison

Dimension	A Intent	B Feedback	C Gap
Developer utility	interpretation provenance	correction effects	blocking prerequisites
Core algorithm	evidence-linked extraction	event correlation + versioning	DAG evaluation
Required data	request + historical context	decision + correction	workflow requirements + context
Feedback	inspection of edits	first-class confirmed updates	clarification resolution
Six-hour slice	high	medium	high
Product relevance	quoting	all four workflows	export/import/booking

24. Shared Component Reuse

Shared once: EventEnvelope, SourceRef, ContextField, SemanticRelation, field version store, tenant authorization, adapter ports, structured extractor, state transition validator, audit projection and React inspection shell. A adds reference resolver and interpretation trace; B adds correction correlation, confirmation and decision diff; C adds workflow dependency templates, DAG traversal and clarification generator.

25. Integration Roadmap

Horizon 1, six-hour hackathon: synthetic events, SQLite, mocked agent and TMS, one complete B cycle with A/C projections. Horizon 2, estimated 2-4 engineering weeks for a limited pilot after access and contract alignment: add read-only email/TMS adapters, real decision hooks, tenant controls and evaluation harness. Horizon 3, estimated 6-12 additional weeks depending on system APIs and operational approvals: cross-workflow entity links, PostgreSQL, background workers, longitudinal metrics, approval routing and replay observability. Estimates are planning assumptions, not commitments.

26. Recursive Architecture Review

R1: tenant/user scope explicitly modeled. R2: all interaction categories have event types. R3: relations carry evidence and temporal validity. R4: decisions are snapshotted and linked to retrieval manifests. R5: corrections remain separate from confirmation. R6: only confirmed updates affect later retrieval. R7: superseded versions remain replayable. Inverse verification of demo A requires parser, extractor, resolver, trace API and UI; demo B requires decision snapshot, feedback correlation, confirmation, retrieval, comparison and timeline; demo C requires dependency DAG, availability checker, clarification endpoint and re-evaluation.

Open dependencies: actual nemox API contracts, availability of decision hooks, TMS record identifiers, approval semantics, tenant identity provider, retention policy and permission to process operational data. These require joint engineering discovery before production integration.

27. Technical Recommendations

Freeze source-linked event and field contracts first. Build one vertical three-cycle demonstration with immutable snapshots and explicit confirmation. Make every interpretation inspectable as a graph edge with provenance. Keep model-generated candidates outside authoritative workflow state. Instrument context retrieval so later behavior can be attributed to a particular confirmed relation. Integrate read-only first, then evaluate controlled operational writes separately.

APPENDIX A | EIGHTEEN MERMAID ARCHITECTURE DIAGRAMS

Diagram 1. nemox System Context

Purpose: nemox system context; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 U[Logistics User] --> E[Email RFQ]
 E --> N[nemox AI Agents - public]
 N --> W[Quotation Export Import Accounting]
 W <--> T[TMS - public]
 N --> D[Developer Observation Adapter - proposed]
 D --> F[Context Field Store - proposed]
 F --> I[Developer Inspector - proposed]
```

Diagram 2. Shared Field Architecture

Purpose: shared field architecture; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 I[Input] --> E[Event Ingestion] --> F[Field Construction] --> S[Semantic Reconstruction] --> R[
Requirements] --> D[Decision Trace] --> B[Feedback] --> V[Field Version] --> C[Context Retrieval] --
> X[Inspection]
 V <--> P[(Persistent Store)]
 E <--> A[External Adapters]
```

Diagram 3. Field Boundary Diagram

Purpose: field boundary diagram; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart TB
 U[User Context] --> C[Customer Context]
```

```
 C --> S[Shipment Context]
 S --> W[Workflow Context]
 W --> A[Agent Execution Context]
 A --> D[Developer Inspection Context]
 X[External Systems] --> W
```

Diagram 4. Field Coupling Diagram

Purpose: field coupling diagram; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 R[Customer Request Field] <-->|references| S[Shipment Field]
 S <-->|requires| C[Carrier Capability Field]
 D[Decision Field] <-->|corrected by| F[Feedback Field]
 F -->|retrieved for| N[Future Interaction Field]
```

Diagram 5. Intent Inspector Components

Purpose: intent inspector components; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 I[Input Adapter] --> P[Parser] --> E[Extractor] --> R[Resolver] --> G[Relation Builder] --> T[
Trace Service] --> U[Dashboard]
```

Diagram 6. Intent Inspector Sequence

Purpose: intent inspector sequence; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
sequenceDiagram
 participant U as User
 participant A as Adapter
 participant R as Resolver
 participant I as Inspector
 U->>A: RFQ
 A->>R: parsed requirements
 R->>I: evidence and unresolved refs
 I->>U: inspection view
```

Diagram 7. Feedback Engine Components

Purpose: feedback engine components; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 D[Decision Collector] --> S[Snapshot Store]
 F[Feedback Processor] --> R[Relation Update]
 R --> V[Version Store]
 V --> C[Context Retrieval] --> X[Decision Comparison] --> T[Timeline]
```

Diagram 8. Recursive Feedback Sequence

Purpose: recursive feedback sequence; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
sequenceDiagram
 participant A as Agent
 participant U as User
 participant F as Feedback Engine
 participant C as Context Store
 A->>F: decision snapshot
 U->>F: correction
 F->>C: proposed relation
 U->>F: confirm
 F->>C: confirmed version
 A->>C: next retrieval
 C-->>A: confirmed relation
```

Diagram 9. Feedback State Machine

Purpose: feedback state machine; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
stateDiagram-v2
 [*] --> INITIAL
 INITIAL --> ENRICHED
 ENRICHED --> UNRESOLVED
 UNRESOLVED --> ENRICHED
 ENRICHED --> CORRECTED
 CORRECTED --> CONFIRMED
 ENRICHED --> CONFIRMED
 CONFIRMED --> SUPERSEDED
```

Diagram 10. Gap Detector Components

Purpose: gap detector components; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 R[Dependency Resolver] --> A[Availability Checker] --> G[Gap Engine] --> Q[Clarification
Generator] --> C[Completion Service] --> I[Gap Inspector]
```

Diagram 11. Gap Dependency Graph

Purpose: gap dependency graph; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart TD
 B[Book Shipment] --> S[Shipment Reference]
 B --> C[Carrier Option]
 B --> D[Delivery Deadline]
 C --> R[Rate Availability]
 D --> H[Historical Constraints]
```

Diagram 12. Gap Resolution Sequence

Purpose: gap resolution sequence; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
sequenceDiagram
 participant U as User
 participant G as Gap Engine
 participant Q as Clarifier
 participant F as Field Store
 G->>Q: blocking attributes
 Q->>U: clarification
 U->>F: sourced answer
 F->>G: updated field
 G-->>U: completion status
```

Diagram 13. Shared Data Architecture

Purpose: shared data architecture; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
erDiagram
 TENANT ||--o{ FIELD : owns
 FIELD ||--o{ RELATION : contains
 FIELD ||--o{ FIELD_VERSION : versions
 EVENT ||--o{ FIELD_VERSION : causes
 DECISION ||--o{ FEEDBACK : receives
 DECISION ||--o{ RETRIEVAL : uses
```

Diagram 14. Event Processing Architecture

Purpose: event processing architecture; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 I[POST Event] --> K[Idempotency Check] --> E[(Append Only Events)] --> O[(Outbox)] --> P[
```

```
Projector] --> V[(Field Versions)]
 P --> X[Inspection API]
```

Diagram 15. AI Integration Architecture

Purpose: ai integration architecture; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 S[Source Segments] --> T[Prompt Template] --> A[Model Adapter] --> J[Structured JSON] --> V[
Pydantic Validation] --> E[Evidence Check] --> F[Candidate Field]
 V -->|invalid| R[Bounded Retry] --> A
```

Diagram 16. Cross-Workflow Context Architecture

Purpose: cross-workflow context architecture; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart LR
 C[Customer] --> Q[Quote] --> B[Booking] --> S[Shipment] --> I[Invoice] --> F[Feedback]
 F --> R[Confirmed Relation Store]
 R --> Q
```

Diagram 17. Two-Developer Task Dependency Graph

Purpose: two-developer task dependency graph; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
flowchart TD
 C[Shared Contract] --> A[Dev A Backend]
 C --> B[Dev B Frontend]
 A --> I[Integration]
 B --> I
 I --> T[Joint Tests] --> D[Demo]
```

Diagram 18. Six-Hour Implementation Timeline

Purpose: six-hour implementation timeline; nodes denote proposed components unless explicitly identified as public product capabilities.

```mermaid
gantt
 title Six Hour Hackathon
 dateFormat HH:mm
 axisFormat %H:%M
 section Shared
 Contracts : 09:00, 1h
 section Developer A
 Backend : 10:00, 2h
 Feedback : 12:00, 2h
 section Developer B
 Frontend : 10:00, 3h
 Integration : 13:00, 1h
 section Joint
 Tests and Demo : 14:00, 1h
```

APPENDIX B | JSON SCHEMAS

```
{ "$schema":"https://json-schema.org/draft/2020-12/schema",
  "title":"ContextField", "type":"object",
  "required":["field_id","scope","scope_ref","version","state","source_refs"],
  "properties":{
    "field_id":{"type":"string","format":"uuid"},
    "scope":{"enum":["user","customer","shipment","agent","workflow","external","developer"]},
    "scope_ref":{"type":"string","minLength":1},
    "version":{"type":"integer","minimum":1},
    "state":{"enum":["INITIAL","ENRICHED","UNRESOLVED","CORRECTED","CONFIRMED","SUPERSEDED"]},
    "source_refs":{"type":"array","items":{"type":"string"}}
  },"additionalProperties":false }

{ "title":"FeedbackEvent", "type":"object",
  "required":["feedback_id","decision_id","actor_id","new_value","source_refs"],
  "properties":{
    "feedback_id":{"type":"string","format":"uuid"},
```

```
    "decision_id":{"type":"string","format":"uuid"},
    "actor_id":{"type":"string"},
    "old_value":{},"new_value":{},
    "explicit_reason":{"type":["string","null"]},
    "source_refs":{"type":"array","minItems":1,"items":{"type":"string"}}
  }}
```

The canonical DTO table in Section 12 defines the remaining records; implement matching JSON Schema generation from versioned Pydantic models rather than manually maintaining divergent copies.

APPENDIX C | API CONTRACT EXAMPLES

```
POST /v1/feedback
Idempotency-Key: synthetic-correction-002
{ "feedback_id":"6fddfb5d-f2f2-40a1-8ac5-5c47c0f621b1",
  "decision_id":"c99a7e45-5b60-4cf1-9d3d-7c67ab4f6e12",
  "actor_id":"operator-1", "field":"carrier_preference",
  "old_value":"Carrier A", "new_value":"Carrier B",
  "source_refs":["event:user-edit-002"] }
201 Created
{ "candidate_relation_id":"rel-002", "status":"PROPOSED",
  "confirmation_required":true, "schema_version":"1.0" }
```

APPENDIX D | REPRESENTATIVE CODE FRAGMENTS

```
def validate_transition(current, next_state):
    allowed = {
      'INITIAL': {'ENRICHED'},
      'ENRICHED': {'UNRESOLVED','CORRECTED','CONFIRMED'},
      'UNRESOLVED': {'ENRICHED'},
      'CORRECTED': {'CONFIRMED'},
      'CONFIRMED': {'SUPERSEDED'},
      'SUPERSEDED': set(),
    }
    if next_state not in allowed[current]:
        raise InvalidTransition(current, next_state)
```

APPENDIX E | IMPLEMENTATION TASK MATRIX

Workstream	Owner	Input	Done when
Contracts + fixtures	Both	three synthetic cycles	DTOs frozen
Event kernel	A	DTOs	replay deterministic
A extractor/resolver	A	source segments	trace source-linked
B feedback/correlation	A	decision + edit	confirmed relation retrieved
C dependency DAG	A	required attributes	minimal gaps computed
Developer dashboard	B	OpenAPI + fixture	three views navigable
Integration + demo	Both	seeded dataset	replay from clean DB

APPENDIX F | TEST SCENARIOS

Scenario A: input references last week shipment and two-day earlier delivery; expected unresolved historical reference until disambiguation and a temporal delta tied to the original sentence. Scenario B: three-cycle carrier correction; expected original snapshot unchanged, candidate update separate, confirmed version retrieved and decision diff rendered. Scenario C: cheaper shipment with missing deadline; expected a minimal clarification question and completed field only after sourced response. Negative fixtures: cross-tenant references, malformed model JSON, duplicate events, invalid state edge, out-of-order feedback, stale external record, ambiguous shipment match.

APPENDIX G | SOURCE REFERENCES

[S1] nemox, product website, https://nemox.ai/, observed 2026-11-01. Company-described quotation, export, import, accounting, email/TMS, human review and logging.

[S2] nemox, About, https://nemox.ai/about, observed 2026-11-01. Company background and founders.

[S3] nemox, Founding Engineer, https://nemox.ai/de/careers/founding-engineer, observed 2026-11-01. Engineering focus on AI context and accuracy.

[S4] nemox, Setup & System Requirements, https://nemox.ai/setup, observed 2026-11-01. Public Microsoft 365 connection and mailbox permission description.

[S5] Technical University of Munich, Project Study @ nemox, dated 2026-10-21, https://www.ie.mgt.tum.de/en/ent/news/news-single-view/article/project-study-nemox-how-do-you-sell-ai-to-a-traditional-industry/. Public description of inbox workflows.

[S6] LOGISTIK HEUTE, Logistics IT: AI Assistant for Medium-Sized Businesses Works in Outlook Mailbox, 2026-09-28, https://logistik-heute.de/en/news/logistics-it-ai-assistant-medium-sized-businesses-works-outlook-mailbox-285674.html. Independent reporting.

Source status: public descriptions were consulted; no private nemox architecture, repository, API implementation or operational telemetry was inspected. All field services, DTOs, algorithms, diagrams and roadmap estimates in this document are proposed reference architecture.